# Asynchronous Multi-Party One-Time Pad Protocols based on -Technique Name-

Giovanni Di Crescenzo[1] Aman N. S.[2]

[1] Peraton Labs, Basking Ridge, NJ, USA
E-mail: gdicrescenzo@peratonlabs.com
New York University, New York, NY, USA
E-mail: gdc258@nyu.edu

[2] New York University, New York, NY, USA
E-mail: aas10498@nyu.edu

**Abstract.** OLD - TO BE UPDATED...
The "One-Time Pad" is a fundamental cryptographic protocol as it represents the ideal in secure unidirectional communication (i.e., in cases where there is a designated sender and a designated receiver) both in terms of security (in the presence of eavesdroppers) as well as in terms of computational efficiency. Surprisingly, no modeling and investigation of this protocol has been done in important practical settings, as distributed and asynchronous ones. In this work we introduce an asynchronous model for multidirectional and multi-player One-Time Pad asynchronous communication protocols. In this model the random pad is shared by all players, and there is no designated sender and receiver; in fact any participating player can act as a receiver at any given time, players communicate in a totally asynchronous fashion and may arbitrarily go off-line. We define the problem of designing One-Time Pad asynchronous communication protocols, where the goal is that of maximizing the amount of the shared pad used before new randomness needs to be generated, with the constraint of mantaining the security property under reasonable adversarial assumptions on the relative behavior of the players and the network. We present lower bounds and protocol solutions for this problem that significantly improve over the obvious scenario where parties use an equal fraction of the pad. Our constructions are *non-interactive* in the sense that they require no additional synchronizing communication beyond the (usual) information that accompanies each ciphertext.

**Keywords.** Private-Key Encryption, One-time Pad, Asynchronous Communication, Group Communication

## 1 Introduction

OLD - TO BE UPDATED...

## 2 Definitions and Preliminary Results

OLD - TO BE UPDATED...

We start a formal investigation of the problem of using one-time pads in the asynchronous setting. Specifically, in this section we present definitions for the communication model and protocols, the security requirements as well as an efficiency metric, which we instantiate through examples of basic protocols.

**OTPAC Protocols.** In our setting, $m$ players $A_1, \ldots, A_m$ wish to communicate securely taking advantage of a shared random pad. Players are assumed to communicate through a network that supports broadcast transmission and each player can be a sender or a receiver in a certain communication. There is no global clock that the players can use for synchronization and message delays are unbounded; all messages are eventually delivered but their order is not necessarily preserved by the network (such assumptions are typical in the Distributed Computing literature). Each player $A_i$ is initialized by executing an Initialization procedure, and subsequently sequences of two events may happen to $A_i$: Send and Receive, according to the requests of sending messages that the specific application environment demands from all players in the protocol. Each player $A_j$ is assumed to have access to the same shared private *random pad* $\boldsymbol{p}$ that can be written as the concatenation of $n$ words $p_1, \ldots, p_n$, of $k$ bits each, $k$ being the length of one message-block. The communication is driven by a sequence of *send requests* for the players (that can be thought to be adversarially generated) and are ordered according to some (unknown to the players) global notion of time; the send requests form the *schedule* of the protocol denoted by $\boldsymbol{s} = ((s_1, t_1), \ldots, (s_v, t_v))$, where $t_1 \leq t_2 \leq \cdots \leq t_v$ correspond to the time of the send requests and each $s_\ell = (j, u)$ stands for the $u$-th request of the $j$-th player. To achieve the highest possible generality we will assume that each execution of the protocol takes as input a pad $\boldsymbol{p}$ and follows an arbitrary schedule $\boldsymbol{s}$. Let us denote by $[n] = \{1, \ldots, n\}$. We can now proceed with a formal definition.

**Definition 1.** Let $m, n$ be positive integers, let $\mathsf{Head} = [m] \times [n]$, and let Conf denote a set of possible configurations; at local time $t$ we denote the configuration of player $A_j$ as $\Pi_j[t] \in \mathsf{Conf}$. We define a *One-Time-Pad Asynchronous Communication* (OTPAC) protocol as a triple of polynomial-size circuits $\langle I, S, R \rangle$, with the following functionality:

$$I : [m] \to \mathsf{Conf} \quad S : [m] \times \mathsf{Conf} \to \{\perp\} \cup (\mathsf{Conf} \times [n]) \quad R : [m] \times \mathsf{Conf} \times \mathsf{Head} \to \mathsf{Conf}$$

The operation of an OTPAC protocol is as follows: initially every player $A_j$ executes $I(j) \to \Pi_j[0]$ to set its initial configuration. Then the sequence of send requests in the schedule $\boldsymbol{s}$ is sequentially scanned. Given a send request at time $t$, player $A_j$ generates a Send event, by executing $S(\Pi_j[t]) \to \langle \Pi_j[t+1], i \rangle$ and resulting in the transmission of the ciphertext $\langle j, i, p_i \oplus M \rangle$, where $M \in \{0,1\}^k$ is the message that player $A_j$ wishes to transmit and $\langle j, i \rangle \in [m] \times [n]$ is the ciphertext header (note that frequently we will write $S(\cdot)$ instead of $S(j, \cdot)$ if the sending player is clear from the context; similarly for $R(\cdot)$). If $S(\Pi_j[t]) = \perp$

player $A_j$ ceases to send messages. Every ciphertext has length $\log m + \log n + k$. A Receive event is triggered by the arrival of a ciphertext $\langle j', i', C\rangle$ at player $A_j$ in time $t$. Player $A_j$ decrypts the message by computing $p_{i'} \oplus C$ and updates its configuration to $\Pi_j[t+1] \leftarrow R(\Pi_j[t], j', i')$. We assume that the $S, R$ circuits satisfy the property $S(\Pi_j[t]) = \perp \Rightarrow S(R(\Pi_j[t], j, i)) = \perp$ for any $\langle j, i\rangle \in \mathsf{Head}$. An OTPAC protocol stops when all parties $A_j$ enter in a configuration $\Pi_j[t]$ such that $S(\Pi_j[t]) = \perp$. A particular execution of an OTPAC protocol (namely, an execution of algorithms $I, S, R$, for fixed random tapes of all participants) can be characterized as in the following definition.

**Definition 2.** A feasible *asynchronous communication pattern* (ACP) for an $m$-player OTPAC protocol $\langle I, S, R\rangle$ on input an $n$-word pad $\boldsymbol{p}$ and an $nm$-request schedule $\boldsymbol{s}$ is a finite directed acyclic graph $\mathcal{P}$ so that the set of its nodes is a subset of $\mathbb{N} \times \mathbb{N}$ and is partitioned into $m$ subsets. Every node $\langle j, t\rangle$ is labeled by an element of $\mathsf{Conf}$, denoted by $\Pi_j[t]$. $\mathcal{P}$ includes the nodes $\langle 1, 0\rangle, \ldots, \langle m, 0\rangle$ labeled by $I(1), \ldots, I(m)$ respectively, as well as the nodes $\langle 1, t_{1,\max}\rangle, \ldots, \langle m, t_{m,\max}\rangle$ such that for any $t > t_{j,\max}$, $\langle j, t\rangle \notin \mathcal{P}$. For any $j, t < t_{j,\max}$ there is an edge from $\langle j, t\rangle$ to $\langle j, t'\rangle$ for some $t'$ with $t' > t$ and such that for all $t^* \in \{t+1, \ldots, t'-1\}$, $\langle j, t^*\rangle \notin \mathcal{P}$; the node $\langle j, t'\rangle$ is the *subsequent* node of $\langle j, t\rangle$. The nodes of $\mathcal{P}$ and their labels fall into either one of the following three categories:

- *Send Nodes.* It is a node $\langle j, t\rangle$ with $S(\Pi_j[t]) \neq \perp$ that has $m$ outgoing edges to $\langle 1, t_1\rangle, \ldots, \langle m, t_m\rangle$ s.t. $t_j = t + 1$ and at most one incoming edge. The label of $\langle j, t+1\rangle$ is equal to $\Pi$ where $\langle \Pi, i\rangle \leftarrow S(\Pi_j[t])$.
- *Receive Nodes.* It is a node $\langle j, t\rangle$ with one incoming edge coming from some Send-node $\langle j', t'\rangle$ $j' \neq j$. The label of $\langle j, t+1\rangle$ is equal to $R(\Pi_j[t], j', i')$ where $i'$ is such that $\langle \Pi', i'\rangle \leftarrow S(\Pi_{j'}[t'])$.
- *Idle Nodes.* It is a node $\langle j, t\rangle$ that is neither a Send or Receive node. If $t > 0$, the node has one incoming and one outgoing edge to its subsequent node $\langle j, t'\rangle$ (recall $t' > t$); if $\langle j, 0\rangle$ is idle then it has no incoming edge. The label of $\langle j, t'\rangle$ is equal to $\Pi_j[t]$.

An example of an ACP is given in figure **??**. Any OTPAC protocol $\mathcal{A} := \langle I, S, C\rangle$ for $m$ players using a random pad $\boldsymbol{p}$ of $n$ $k$-bit words and an $nm$-request schedule $\boldsymbol{s}$ defines a family of feasible ACP's denoted by $\mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$. (By $\mathcal{F}^{m,n}_{\mathcal{A}}$ we denote the union, over all schedules $\boldsymbol{s}$, of the family of feasible ACP's $\mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$.)

**Security and efficiency of OTPAC protocols**. We define the security of OTPAC protocols by extending the security notion of the one-time pad from the unidirectional synchronous setting (namely, that the same portion of the pad should not be used twice) to the asynchronous setting considered here.

Let $\mathcal{A}$ be an OTPAC protocol, and $\mathcal{F}^{m,n}_{\mathcal{A}}$ be the set of feasible ACP's. While players $A_1, \ldots, A_m$ are attempting to communicate, the adversary is capable of controlling the network so to both choose a particular schedule of send requests and swap or delay messages being sent. The goal of the adversary is to force the re-use of some portion of the pad so that the "one-time" property is lost and some information can be derived from the transcript. It follows that the only mechanism available to players to protect the security of their communication

is to reach the $\perp$ output of their Send-event state update function $S$, before the adversary is capable of violating the one-time property. So given an OTPAC protocol $\mathcal{A}$, the adversary selects a schedule $\boldsymbol{s}$ that is input to $\mathcal{A}$, and furthermore selects a pattern $\mathcal{P} \in \mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$; the protocol is executed following the pattern $\mathcal{P}$; if $\mathcal{P}$ contains two Send events that use the same pad segment, then the adversary wins. In conclusion, security of an OTPAC protocol would be argued for a subfamily of $\mathcal{F}^{m,n}_{\mathcal{A}}$ for which no pad is used twice. Formally,

**Definition 3. The Security Property.** Let $\mathcal{A} := \langle I, S, C\rangle$ be an OTPAC protocol, let $\mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$ the be family of feasible ACP's for $\mathcal{A}$ on input schedule $\boldsymbol{s}$ and let $\mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}} = \cup_{\boldsymbol{s}}\mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$. We say that $\mathcal{A}$ is *insecure on* $\mathcal{P} \in \mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$ if $\mathcal{P}$ includes two Send nodes $\langle j, t\rangle$ and $\langle j', t'\rangle$ so that if $\langle \Pi, i\rangle \leftarrow S(\Pi_j[t])$ and $\langle \Pi', i'\rangle \leftarrow S(\Pi_{j'}[t'])$ it holds that $i = i'$. Let $\mathcal{G} = \cup_{\boldsymbol{s}}\mathcal{G}_{\boldsymbol{s}}$ be a family of ACP's, where for each $\boldsymbol{s}$, $\mathcal{G}_{\boldsymbol{s}} \subseteq \mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$. We say that $\mathcal{A}$ is *secure on family* $\mathcal{G}$, if there exists no schedule $\boldsymbol{s}$ and no pattern $\mathcal{P} \in \mathcal{G}_{\boldsymbol{s}}$ such that $\mathcal{A}$ is insecure on $\mathcal{P}$.

Naturally, for any given $\mathcal{A}$ it would be the most desirable that it is secure for any $\mathcal{P} \in \mathcal{F}^{m,n}_{\mathcal{A}}$. Considering only such protocols though is very restrictive. Instead, as it is standard in cryptography as well as in distributed systems theory, we will opt to restrict the behavior of the adversary in a meaningful ways and consider protocols that can be proven secure in such adversarial setting.

**Adversarial Setting: Undelivery.** A communication pattern $\mathcal{P} \in \mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$ admits various total orderings $\prec$ between the vertices of $\mathcal{P}$ (topological sortings of the graph $\mathcal{P}$). One such sorting $\prec$ corresponds to the actual "real global time" (unknown to the participants). Such total orderings can also be called "runs" of an asynchronous communication pattern $\mathcal{P}$, following the standard terminology in distributed systems. Note that, as it is common in distributed systems theory, we assume that no two events happen at the same time, i.e. the total ordering $\prec$ is strict. A Send event $\langle j, t\rangle$ is said to have $d$ *undelivered messages* w.r.t. a topological sorting $\prec$ of $\mathcal{P}$, if there exist $d$ Send events $\langle j_1, t_1\rangle, \ldots, \langle j_d, t_d\rangle$, $j \notin \{j_1, \ldots, j_d\}$ for which it holds $\langle j_\ell, t_\ell\rangle \prec \langle j, t\rangle$ for $\ell = 1, \ldots, d$, and if $\langle j, s_1\rangle, \ldots, \langle j, s_d\rangle$ are the corresponding Receive events at player $A_j$ it holds that $s_\ell > t$ for $\ell = 1, \ldots, d$. For any schedule $\boldsymbol{s}$, let $\mathcal{U}^{m,n}_{\mathcal{A},\boldsymbol{s},d}$ denote the subfamily of all ACP's $\mathcal{P} \in \mathcal{F}^{m,n}_{\mathcal{A},\boldsymbol{s}}$ that have no Send events with $d$ undelivered messages w.r.t. any possible topological sorting of $\mathcal{P}$. Also, let $\mathcal{U}^{m,n}_{\mathcal{A},d} = \cup_{\boldsymbol{s}}\mathcal{U}^{m,n}_{\mathcal{A},\boldsymbol{s},d}$.

Note that undelivery does not describe any fixed property of the network. More specifically, if an ACP contains a Send event that has $d$ undelivered messages this does not mean that the network delays messages by a fixed upper time-bound that is proportional to $d$. The way to interpret an ACP that contains a Send-event with $d$ undelivered messages is that the adversary controlled the message delivery of the network and the random coin tosses of the players' random processes in such a way so that one of the players initiated a Send-event before having heard of $d$ Send events of other players that happened concurrently. We remark that if it was possible for the players to synchronize themselves (e.g., by using a global clock) it would be easy to restrict the adversary to produce only ACP's that belong to $\mathcal{U}^{m,n}_{\mathcal{A},1}$, i.e., if a player sends a message the other players

are silent until they receive it. Clearly in such a scenario designing an OTPAC protocol is a trivial matter. Tackling the asynchronous case where the parameter $d > 1$ is the motivation for our investigations.

**Efficiency of OTPAC protocols.** We introduce a natural metric of the efficiency of an OTPAC protocol with respect to usage of the shared pad vector.

We start with some definitions. The *message-count* of an ACP $\mathcal{P} \in \mathcal{F}_{\mathcal{A}}^{m,n}$, denoted by $\mathsf{mc}(\mathcal{P})$, is a tuple $\langle N_1, \ldots, N_m \rangle$ that denotes the number of $\mathsf{Send}$ events for each of the players $A_1, \ldots, A_m$. Let $\mathcal{G} = \cup_{\boldsymbol{s}} \mathcal{G}_{\boldsymbol{s}}$ be a family of ACP's, where for each $\boldsymbol{s}$, $\mathcal{G}_{\boldsymbol{s}} \subseteq \mathcal{F}_{\mathcal{A},\boldsymbol{s}}^{m,n}$. Next we define the *order* of a family $\mathcal{G} \subseteq \mathcal{F}_{\mathcal{A}}^{m,n}$. Intuitively, the order of $\mathcal{G}$ is the maximum number of messages that all players can send using the OTPAC protocol $\mathcal{A}$ when, on input schedule $\boldsymbol{s}$, the communication pattern is drawn from $\mathcal{G}_{\boldsymbol{s}}$. Formally, we define the *order* of $\mathcal{G}$ as $N = \min_{\boldsymbol{s}} \min_{\mathcal{P} \in \mathcal{G}_{\boldsymbol{s}}} \sum_{i=1}^{n} N_i$. Using this notion, we can define our main metric for OTPAC protocols, which we call the "efficiency ratio," standing for the maximum percentage of the pad all players are allowed to use.

**Definition 4. The Efficiency Ratio**. Let $\mathcal{A}$ be an OTPAC protocol with corresponding ACP family $\mathcal{F}_{\mathcal{A}}^{m,n}$ and let $\mathcal{G} = \cup_{\boldsymbol{s}} \mathcal{G}_{\boldsymbol{s}}$ be an ACP family such that for each $\boldsymbol{s}$, $\mathcal{G}_{\boldsymbol{s}} \subseteq \mathcal{F}_{\mathcal{A},\boldsymbol{s}}^{m,n}$. The *efficiency ratio of algorithm $\mathcal{A}$ on family $\mathcal{G}$* is defined as $\alpha := \frac{N}{n}$ where $N$ is the order of family $\mathcal{G}$.

Although it does not appear in the above definition, we stress that this metric only makes sense for secure executions, as insecure executions can always have efficiency ratio 1 or even greater.

In order to understand the above notion it would be helpful to recall the two simple protocols described in the introduction: $\mathcal{A}_1$ will be the OTPAC protocol where all players use a disjoint segment in the pad and $\mathcal{A}_2$ will be the protocol that all players use the whole pad at the same time while maintaining a joint counter. We refer to the appendix if the reader wishes to see the explicit implementation of $\mathcal{A}_1$ and $\mathcal{A}_2$ in the OTPAC model. It is straightforward to show (and for the sake of completeness we do so in Appendix **??**) that $\mathcal{A}_1$ has efficiency ratio $1/m$ and can be proven secure for all feasible ACP's $\mathcal{F}_{\mathcal{A}_1}^{m,n}$, whereas $\mathcal{A}_2$ has efficiency ratio 1, but can be proven secure only for $\mathcal{U}_{\mathcal{A}_2,1}^{m,n}$, the family of ACP's that are effectively synchronous (no $\mathsf{Send}$-event has an undelivered message). The protocols $\mathcal{A}_1, \mathcal{A}_2$ represent the two different ends of the spectrum: $\mathcal{A}_1$ allows arbitrary adversarial action but has very small efficiency ratio; on the other hand $\mathcal{A}_2$ allows the highest efficiency ratio nevertheless it can only be proven secure if we restrict the adversary so much so that players communicate in an effectively synchronous fashion. This motivates our investigations in the rest of the paper, where we will present constructions of OTPAC protocols that trade adversarial action for efficiency.

## 3 Known OTPAC Protocol for Two Parties

This section had OLD material, only included here as writing examples...

### 3.1 The Disjoint-Pads OTPAC Protocol for Two Parties

In order to understand the notion of the efficiency ratio it would be helpful to consider some simple OTPAC protocols. The following OTPAC protocol will be denoted by $\mathcal{A}_1$, and has $\mathsf{Conf} = [n] \cup \{0\}$.
*Informal Description of* $\mathcal{A}_1$. The pad vector $\boldsymbol{p}$ is divided into $m$ equal-size disjoint segments. Every player is assigned to one of these segments, uses pads from her segment only, and halts after she has used all bits in her segment.

Procedure $I$:
  `input:` $id$
  $\mathsf{u} \leftarrow (id-1) * \lfloor \frac{n}{m} \rfloor$;
  `return`($\mathsf{u}$)
Procedure $S$:
  `input:` $\mathsf{u}$
  $\mathsf{u} \leftarrow \mathsf{u} + 1$;
  `if` $\mathsf{u} \leq id * \lfloor \frac{n}{m} \rfloor$ `then return`($\langle \mathsf{u}, \mathsf{u} \rangle$) `else return`($\perp$);
Procedure $R$:
  `input:` $\mathsf{u}$ `and` $\langle id^*, \mathsf{pad} \rangle$
  `return`($\mathsf{u}$)

**Theorem 1.** *The OTPAC protocol* $\mathcal{A}_1$ *is secure for the family* $\mathcal{F}_{\mathcal{A}_1}^{m,n}$ *(all feasible communication patterns of* $\mathcal{A}_1$*). The order of* $\mathcal{F}_{\mathcal{A}_1}^{m,n}$ *is* $\lfloor \frac{n}{m} \rfloor$*. It follows that the efficiency ratio of* $\mathcal{A}_1$ *is* $\alpha = \lfloor \frac{n}{m} \rfloor / n \approx 1/m$.

### 3.2 The No-Undelivery OTPAC Protocol for Two Parties

While $\mathcal{A}_1$ is the best one can hope in terms of security, its efficiency is low. Indeed, each player can only use a fraction of $1/m$ of the shared random pad. Our second example, OTPAC Protocol $\mathcal{A}_2$, is the other end of the spectrum. $\mathcal{A}_2$, with $\mathsf{Conf} = [n] \cup \{0\}$ is described below:
*Informal Description of* $\mathcal{A}_2$. Every player initializes a counter to the beginning of the pad vector $\boldsymbol{p}$. Players use pads as dictated by the counter; upon receiving a message, a player updates its counter appropriately.

Procedure $I$:
  `input:` $id$
  $\mathsf{u} \leftarrow 0$;
  `return`($\mathsf{u}$)
Procedure $S$:
  `input:` $\mathsf{u}$
  $\mathsf{u} \leftarrow \mathsf{u} + 1$;
  `if` $\mathsf{u} \leq n$ `then return`($\langle \mathsf{u}, \mathsf{u} \rangle$) `else return`($\perp$);
Procedure $R$:
  `input:` $\mathsf{u}$ `and` $\langle id^*, \mathsf{pad} \rangle$
  `if` $\mathsf{u} < \mathsf{pad}$ `then` $\mathsf{u} \leftarrow \mathsf{pad}$;
  `return`($\mathsf{u}$)

To argue about the security of $\mathcal{A}_2$ we need to characterize the subfamily of feasible ACP's $\{\mathcal{P} \mid \mathcal{A}_2$ is insecure for $\mathcal{P}\}$.

**Lemma 1.** *Let $\mathcal{P} \in \{\mathcal{P} \mid \mathcal{A}_2$ is insecure for $\mathcal{P}\}$. Then, $\mathcal{P}$ contains a* Send*-event with at least 1 undelivered message.*

*Proof.* Since $\mathcal{A}_2$ is insecure for $\mathcal{P}$ it follows that there are two Send-events $\langle j, t\rangle$ and $\langle j', t'\rangle$ for which it holds that $i = i'$ where $\langle \Pi, i\rangle \leftarrow S(\Pi_j[t])$ and $\langle \Pi', i'\rangle \leftarrow S(\Pi_{j'}[t'])$. Assume towards contradiction that $\mathcal{P}$ contains no Send-event with at least 1 undelivered message. Then all outputs of $S$ contain different pairs $\langle u, u\rangle$, and therefore for $\langle \Pi, i\rangle$, $\langle \Pi', i'\rangle$, it holds that $i \neq i'$, which gives the desired contradiction. □

The theorem below follows:

**Theorem 2.** *The OTPAC protocol $\mathcal{A}_2$ defined above is secure for the family $\mathcal{U}^{m,n}_{\mathcal{A}_2,0}$ and has efficiency ratio* 1*.*

It follows that $\mathcal{A}_2$ is the best that one can hope for in terms of efficiency. Nevertheless the subfamily $\mathcal{U}^{m,n}_{\mathcal{A}_2,0}$ is very restricting in terms of adversarial action. Indeed it implies that no two Send-events occur concurrently, i.e., the ACP's for which $\mathcal{A}_2$ can be proven secure are effectively synchronous.

### 3.3 A Previous Optimal OTPAC Protocol for Two Parties

In this section we show a 2-party OTPAC protocol with optimal efficiency ratio. Fornally, we achieve the following

**Theorem 3.** *The OTPAC protocol $\mathcal{A}_3$ is secure for $\mathcal{U}^{2,n}_{\mathcal{A}_3,d} \subseteq \mathcal{F}^{2,n}_{\mathcal{A}_3}$ The efficiency ratio of $\mathcal{A}_3$ over $\mathcal{U}^{2,n}_{\mathcal{A}_3,d}$ equals $1 - d/n$.*

**Remarks.** Because of Theorem **??**, we obtain that protocol $\mathcal{A}_3$ (resp., $\mathcal{A}_1$) has *optimal* efficiency ratio over family $\mathcal{U}^{2,n}_{\mathcal{A}_3,d+1} \subseteq \mathcal{F}^{2,n}_{\mathcal{A}_3}$ for $d \leq \lfloor n/2 \rfloor$ (resp., $d > \lfloor n/2 \rfloor$).

**Informal Description of protocol $\mathcal{A}_3$.** The two players initialize counters, each one at the far end of the pad vector. Players expend pads in a "head-on collision run" to each other while maintaining a "safety distance" from each other's pad segment. The safety distance value is determined by the undelivery parameter $d$. A formal description of the protocol is presented below.

**Formal Description of protocol $\mathcal{A}_3$.** The protocol described in this section will be denoted by $\mathcal{A}_3 = \langle I, S, R\rangle$. The set of configurations is defined as $\mathsf{Conf} = ([n] \cup \{0\}) \times ([n] \cup \{0\})$.

Procedure $I$:
input: $id$
$\mathsf{u} \leftarrow 0$; /*messages sent by me*/
$\mathsf{v} \leftarrow 0$; /*messages sent by other*/

Procedure $S$:
input: $\langle \mathsf{u}, \mathsf{v} \rangle$
$\mathsf{u} \leftarrow \mathsf{u} + 1$;
if $\mathsf{u} + \mathsf{v} \leq n - d$ then
    if $(id = 1)$ then $\mathsf{pad} \leftarrow \mathsf{u}$;
    if $(id = 2)$ then $\mathsf{pad} \leftarrow n + 1 - \mathsf{u}$;
    return$(\langle \mathsf{u}, \mathsf{v} \rangle, \mathsf{pad})$;
else return$(\perp)$;

Procedure $R$:
input: $\langle \mathsf{u}, \mathsf{v} \rangle$ and $\langle id^*, \mathsf{pad} \rangle$
if $(id^* = 1)$ then $\mathsf{u}^* \leftarrow \mathsf{pad}$;
if $(id^* = 2)$ then $\mathsf{u}^* \leftarrow n + 1 - \mathsf{pad}$;
if $\mathsf{u}^* > \mathsf{v}$ then $\mathsf{v} \leftarrow \mathsf{u}^*$;
return$(\langle \mathsf{u}, \mathsf{v} \rangle)$

We start the proof of the security property by stating and proving a lemma which characterizes the ACP's for which the security of $\mathcal{A}_3$ fails.

**Lemma 2.** *Let $\mathcal{P} \in \mathcal{F}_{\mathcal{A}_3}^{m,n}$ for which $\mathcal{A}_3$ is insecure. Then, $\mathcal{P}$ has* Send*-event with at least $d + 1$ undelivered messages.*

*Proof.* The proof actually shows a somewhat stronger characterization than what stated in the lemma.

Let $\mathcal{P} \in \mathcal{F}_{\mathcal{A}}^{m,n}$. A *mutual-delay-pattern* of order $d$ is a subgraph of $\mathcal{P}$ that has $4d$ vertices and $2d$ edges, so that $\langle j, t_1 \rangle, \ldots, \langle j, t_d \rangle$, and $\langle j', t'_1 \rangle, \ldots, \langle j', t'_d \rangle$ are Send events, with $j \neq j'$, with corresponding Receive events, $\langle j', s'_1 \rangle, \ldots, \langle j', s'_d \rangle$ and $\langle j, s_1 \rangle, \ldots, \langle j, s_d \rangle$, respectively. Additionally we require that $\max\{t_1, \ldots, t_d\} < \min\{s_1, \ldots, s_d\}$ and $\max\{t'_1, \ldots, t'_d\} < \min\{s'_1, \ldots, s'_d\}$.

We will prove the stronger statement that the OTPAC protocol $\mathcal{A}_3$ being insecure for $\mathcal{P}$ implies that $\mathcal{P}$ contains a mutual delay pattern of order $d + 1$.

Since $\mathcal{A}_3$ is insecure for $\mathcal{P}$, there exist a Send-event $\langle A, t^A \rangle$ with header $\langle A, i^A \rangle$ and a Send-event $\langle B, t^B \rangle$ with header $\langle B, i^B \rangle$, so that $i^A = i^B$. Let $\mathsf{u}[A, t]$ denote the value of the counter $\mathsf{u}$ of player A after the state-update of the event $\langle A, t \rangle$. Similarly we define $\mathsf{u}[B, t], \mathsf{v}[A, t]$ and $\mathsf{v}[B, t]$. Observe now that $i^A = \mathsf{u}[A, t^A]$ and $i^B = n + 1 - \mathsf{u}[B, t^B]$ and as a result $\mathsf{u}[A, t^A] + \mathsf{u}[B, t^B] = n + 1$.

Since at $\langle A, t^A \rangle$ we have a Send event, it follows that $\mathsf{u}[A, t^A] + \mathsf{v}[A, t^A] \leq n - d$. It follows that $n + 1 - \mathsf{u}[B, t^B] + \mathsf{v}[A, t^A] \leq n - d \Longrightarrow \mathsf{u}[B, t^B] \geq \mathsf{v}[A, t^A] + d + 1$. Because $\mathsf{v}[A, t^A] \geq 0$ it follows that $\mathsf{u}[B, t^B] \geq d + 1$. Using a dual argument we also obtain that $\mathsf{u}[A, t^A] \geq d + 1$.

The above two facts suggest that player A before the Send-event $\langle A, t^A \rangle$ has sent at least $d$ messages, and similarly player B before the Send-event $\langle B, t^B \rangle$ has sent at least $d$ messages.

It follows that there are $d+1$ consecutive Send events for player A at local times $t_1^A, \ldots, t_d^A, t^A$ and $d+1$ consecutive Send events for player B at local times $t_1^B, \ldots, t_d^B, t^B$. Observe that $\mathsf{u}[A, t_\ell^A] = \mathsf{u}[A, t^A] + \ell - d - 1$ and $\mathsf{u}[B, t_\ell^B] = \mathsf{u}[B, t^B] + \ell - d - 1$ for $\ell = 1, \ldots, d$.

If $t_\ell^A$ is the local time of a Send-event for player A we denote by $s_\ell^B$ the local time at player B of the corresponding Receive-event. Similarly we define $s_\ell^A$ (for $\ell = 1, \ldots, d$). Similarly denote by $\langle B, s^B \rangle$ the Receive-event that corresponds to $\langle A, t^A \rangle$ and by $\langle A, s^A \rangle$ the Receive-event of $\langle B, t^B \rangle$. We will prove

$$t^A < \min\{s_1^A, \ldots, s_d^A, s^A\} \text{ and } t^B < \min\{s_1^B, \ldots, s_d^B, s^B\}$$

Observe that these two inequalities imply that $\mathcal{P}$ contains a mutual delay pattern of order $d+1$. Let us consider the first inequality (the second can be proven in a similar fashion). For the sake of contradiction we assume is false, i.e., there exists $\ell \in \{1, \ldots, d+1\}$ such that $s_\ell^A < t^A$ (where we denote $s_{d+1}^A = s^A$. In other words the Send-event at local time $t^A$ happens after the Receive-event at local time $s_\ell^A$ which corresponds to the Send-event of player B at local time $t_\ell^B$ (again, we denote $t_{d+1}^B = t^B$).

It follows that

$$\mathsf{v}[A, t^A] \geq \mathsf{v}[A, s_\ell^A] \geq \mathsf{u}[B, t_\ell^B] = \mathsf{u}[B, t^B] + \ell - d - 1$$

On the other hand,

$$\mathsf{v}[A, t^A] \leq n - d - \mathsf{u}[A, t^A] = n - d - (n + 1 - \mathsf{u}[B, t^B]) = \mathsf{u}[B, t^B] - d - 1$$

As a result we obtain:

$$\mathsf{u}[B, t^B] + \ell - d - 1 \leq \mathsf{u}[B, t^B] - d - 1 \Leftrightarrow \ell \leq 0$$

which is a contradiction, since $\ell \in \{1, \ldots, d+1\}$. □

We continue with the proof of Theorem 4.

*Proof.* Lemma 2 suggests that the set $\{\mathcal{P} \mid \mathcal{A}_3 \text{ is insecure for } \mathcal{P}\}$ is subsumed into the set of all communication patterns that contain a Send event with at least $d+1$ undelivered messages (actually a mutual delay pattern of order at least $d+1$). Regarding the efficiency ratio of $\mathcal{A}_3$, it is easy to verify that for any schedule $\boldsymbol{s}$, $\mathcal{A}_3$ stops when $A$ and $B$ are $d$ portions of the pad away from each other. It follows that the order of $\mathcal{U}_{\mathcal{A}_3,d}^{2,n}$ is $n-d$ and thus the efficiency ratio of $\mathcal{A}_3$ is $1 - d/n$. □

### 3.4 A Second Optimal OTPAC Protocol for Two Parties

In this section we show another 2-party OTPAC protocol with optimal efficiency ratio. Fornally, we achieve the following

**Theorem 4.** *The OTPAC protocol $\mathcal{A}_4$ is secure for $\mathcal{U}_{\mathcal{A}_4,d}^{2,n} \subseteq \mathcal{F}_{\mathcal{A}_4}^{2,n}$ The efficiency ratio of $\mathcal{A}_4$ over $\mathcal{U}_{\mathcal{A}_4,d}^{2,n}$ equals $1 - d/n$.*

**Informal Description of protocol $\mathcal{A}_4$.** This protocol can be seen as a reformulation of protocol $\mathcal{A}_3$, in that the two parties consume the same sequence of pads as in $\mathcal{A}_3$, but using somewhat different logic to choose the next pad. Player 1 (resp., 2) initializes index 0 (reps., $n+1$) and a direction value equal to $+1$ for 'right' (resp., -1 for 'left'). Each of the two players also initializes a $d$-length tuple, called the *PU buffer*, containing pads along the other player's direction that might have been used but have not been delivered. Upon each send event, a player checks if the next available pad on the player's direction is on the PU buffer; if yes, the player halts; if not, the player uses it and updates the index as the next available pad on the player's direction. Upon each receive event, the player updates the PU buffer, by removing the received index, shifting back all the successor indices and appending one more potentially undelivered index equal to the next pad in the same, other player's, direction. If this appended index is equal to a sent index, then the player halts since no more pads can be safely used. After the update, the PU buffer contains up to $d$ consecutive indices in the other player's direction, following the index of the just received pad. Note that these may be strictly less than $d$ in the event that also one or more pads sent previously to the just received pad, have not yet been delivered.

A formal description of the protocol is presented below.

**Formal Description of protocol $\mathcal{A}_4$.** The protocol described in this section will be denoted by $\mathcal{A}_4 = \langle I, S, R \rangle$.

Procedure $I$:
`input:` $id$
`if` $id = 1$ `then`
  `set` $myind = 0$, $dir = +1$, $PUbuffer = (n, n-1, ..., n-d+1)$
`if` $id = 2$ `then`
  `set` $myind = n+1$, $dir = -1$, $PUbuffer = (1, 2, ..., d)$

Procedure $S$:
`input:` $(myind, dir, PUbuffer)$
$nextind = myind + dir$
`if` $nextind \notin PUbuffer$ `then`
  `set` $myind = nextind$
  `return:` $myind$ `/*` *can send* `*/`
`else return:` $\perp$ `and halt /*` *new pad sequence needed* `*/`

Procedure $R$:
`input:` $(myind, dir, PUbuffer)$ `and` $recind$
`let` $j \in [0, d-1]$ `be such that` $PUbuffer[j] = recind$
`for` $i = j, j+1, \ldots, d-2$,
  `set` $PUbuffer[i] = PUbuffer[i+1]$
`set` $newpuind = PUbuffer[d-1] - dir$
`if` $dir(newpuind - myind) <= 0$ `then`
  `return:` $\perp$ `and halt /*` *new pad sequence needed* `*/`
`set` $PUbuffer[d-1] = newpuind$
`return:` $PUbuffer$ `/*` *buffer updated* `*/`

We start the proof of the security property by stating and proving the main PU buffer property.

**Lemma 3.** *For any $\mathcal{P} \in \mathcal{U}^{m,n}_{\mathcal{A}_4,d}$, when a player's Send event starts in $\mathcal{P}$, if a message corresponding to pad index $ind \in [1,n]$ was already sent and is yet undelivered to that player, then $ind$ is in that player's PU buffer.*

Proof sketch:

A player's Send event can happen after initialization, a Receive event from the same or the other player, or a Send event from the same or the other player.

For the base case, after initialization, the lemma statement is true since the player's PU buffer contains the first $d \geq 1$ indices (and thus the first index) that the other player could use in the protocol.

Assuming the lemma is true up to a given sequence of events, and consider the next Receive or Sent event.

After a Receive event, no new undelivery can happen. If the Receive event is from the other player, the player's PU buffer does not change and so the lemma property is still true. If the Receive event is from the same player, the only PU buffer removed index is the just received index, as this is not undelivered any more, and so the lemma property continues to be true.

After a Send event, an index is used and is still undelivered. If the Send event is from the same party, the PU buffer does not change but the lemma property is still true as no index is undelivered to this party. If the Send event is from the other party, assume that this is the $i$-th Send event from the other party. If $i \leq d$ then the index used was already in the PU buffer after initialization. If $i > d$ then, by definition of $\mathcal{U}^{m,n}_{\mathcal{A}_4,d}$, of the $i-1$ previous Send events from the same party, at least $i-d$ were also received. Each of these $i-d$ Receive events generated a removal of one index and an addition of a new index in the same direction as the other player. This implies that the $i$-th index (in fact, all of the first $i$ indices) in the other player's direction were already entered in the buffer. Since this is the first time that this $i$-th Send event happens, its index was never removed, and so it is currently in the buffer.

We can finally show the proof of the security property of $\mathcal{A}_4$.

**Lemma 4.** *$\mathcal{A}_4$ is secure for family $\mathcal{U}^{m,n}_{\mathcal{A}_4,d}$.*

Proof sketch:

We need to show that for any two successful Send events, one from each party, the associated indices are distinct.

Consider any Send event from party 1 and any Send event from party 2, and assume without loss of generality that the latter happens before the former. We distinguish two cases, depending on whether the index for the Send event from party 2 was delivered or not to party 1 before the Send event from party 1.

Assume the index for Send event from party 2 was not delivered. Then, by Lemma 3, this event's index is in party 1's buffer. By definition of Send, party 1 will not Send any index in the buffer, and so the two indices cannot be equal.

Now, assume the index for the Send event from party 2 was delivered. Because this index was delivered, its Deliver executions can end in one of two ways: either (1) the buffer is updated with a new index being inserted in the buffer, this new index being one step farther in the other player's direction; or (2) the received index is adjacent to a previously sent index, in which case no buffer change happens and a new pad sequence is needed. In case (1), there is at least one buffer index between the two Send events' indices, and thus these indices cannot be equal. In case (2), the two indices are adjacent but still different, and the protocol stops.

## 4 New OTPAC Protocols for $m = 4$ Parties

We now lift the two-party PU-buffer protocol $\mathcal{A}_4$ of Section 3.4 to four parties. Two difficulties appear that are absent for $m = 2$: each player must track *three* other players, any of which may consume the pad word it is about to use; and once two players meet, the free pad lies elsewhere, so a player that still needs to send must *relocate* without colliding with the others. We handle the first difficulty with one reserved buffer per tracked player, maintained as in $\mathcal{A}_4$, and the second with *individual jumps* to the nodes of a fixed binary-space partition (BSP) tree over $[n]$. The resulting protocol is denoted $\mathcal{A}_5$. Its main new feature is that a player's next pad is no longer a function of its own position alone: where it jumps depends on the occupancy of the whole pad, which different players see differently while messages are undelivered. Most of the rules below make the players' predictions of each other's jumps agree.

### 4.1 Informal description of protocol $\mathcal{A}_5$

**Layout and header.** The pad is split into two *arenas*, $[1, \lfloor n/2 \rfloor]$ and $[\lfloor n/2 \rfloor + 1, n]$. Players $A_1, A_2$ start at the two ends of the first arena and $A_3, A_4$ at the two ends of the second; $A_1, A_3$ move right (direction $+1$) and $A_2, A_4$ move left ($-1$). As in $\mathcal{A}_4$, a player uses pad words one at a time in its direction (*walking*); its direction never changes. A ciphertext from $A_K$ carries the header $\langle K, i, c \rangle$, where $i$ is the pad index and $c$ is $K$'s *message number* (its number of Send events so far); this adds $\log n$ bits and is the only change to the model of Section 2.

**Local state.** Player $A_j$ keeps the set $\mathrm{Used}_j$ of pad indices it knows to be used and, for every other player $K$, a believed position $\mathrm{pos}_j(K)$, the highest message number $\mathrm{last}_j(K)$ received from $K$, and a *reserved buffer* $\mathrm{buf}_j(K)$. The buffer plays the role of the PU buffer of $\mathcal{A}_4$: it holds every pad that $K$ may already have used without $A_j$ having heard of it. It consists of a *prediction* $\mathrm{fwd}_j(K)$, the next (at most) $d$ pads $K$ would use from $\mathrm{pos}_j(K)$ as computed by $A_j$, and a set $\mathrm{hold}_j(K)$ of earlier predictions not yet resolved.

**Sending and relocating.** On a Send, $A_j$ uses its next pad if that pad is not used, is nobody's believed position, and is in none of its buffers; otherwise it jumps. The BSP tree splits an interval $[L, R]$ at $M = \lfloor (L + R)/2 \rfloor$ into $[L, M-1]$ and $[M+1, R]$, recursively; each split gives a *node*, the pad pair $\langle M, M+1 \rangle$, and

nodes are listed root first. A right-mover lands on $M+1$ and a left-mover on $M$, so a right- and a left-mover that pick the same node move apart. A jumping player picks the root-most node whose slot is free and that has more than $2d$ free pads on each side; if there is none, it halts. Figure 1 shows a run of the accompanying visualizer in which $A_2$ and then $A_1$ land on the same node.

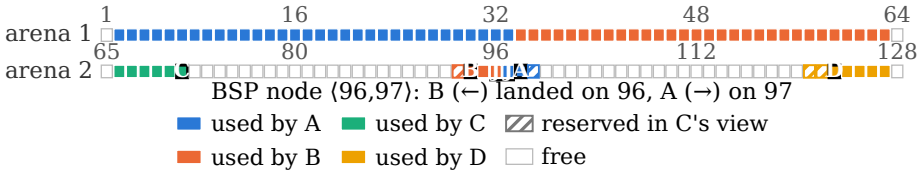



**Fig. 1.** A state of $\mathcal{A}_5$ in the visualizer ($n = 128$, $d = 2$, delivery *worst*, after 78 sends; players $A_1, \ldots, A_4$ are labelled A–D). Arena 1 is full: B walked left to pad 33, could not continue, and jumped to node $\langle 96, 97\rangle$ in arena 2; A followed and landed on the partner slot. Hatched pads are those C reserves for the others, pads they may already have used without C having heard of it; C never uses them.

### 4.2 Rules of $\mathcal{A}_5$

R1 and R2 are the core protocol. Each of R3–R10 fixes a concrete failure that we observed in the simulator.

**R1 (safe send).** A player never uses a pad that is in $\text{Used}_j$, is another player's believed position, or is in one of its buffers.

**R2 (relocation).** When the next pad is refused, the player jumps to the root-most valid BSP node, landing on $M+1$ (right-mover) or $M$ (left-mover).

**R3 (clearance).** A node is valid if the lander's slot is free, the node is not fully occupied, and more than $2d$ pads on each side, outside $M$ and $M+1$, are free. On the lander's back side, pads merely reserved for players moving away from the node do not count as occupied.

**R4 (fresh prediction).** $\text{fwd}_j(K)$ is recomputed from scratch whenever the state changes, never patched.

**R5 (disjoint reservations).** A pad reserved for one player is not reserved for another.

**R6 (valid jumpspots only).** A jump is predicted only to a node that is currently valid.

**R7 (re-validate all).** Every Receive recomputes the predictions for all three tracked players.

**R8 (message order).** Let $c$ be the number of a message from $K$ and $\ell = \text{last}_j(K)$. (a) If $c \le \ell$, only its pad is marked used and $\text{pos}_j(K)$ does not move back. (b) If $c = \ell + 1 + k$ with $k > 0$, the $k$ pads that $K$ used unheard are marked used: the first $k$ predicted pads if the new pad is the $(k{+}1)$-th

predicted one; the $k$ pads in between if $K$ walked straight; otherwise (an unforeseen jump) every pad $K$ could have used, namely the $k$ pads after $\mathrm{pos}_j(K)$ and the $k$ pads before the new pad.

**R9 (held reservations).** A predicted pad stays reserved, in $\mathrm{hold}_j(K)$, until all of $K$'s messages up to the one it was predicted for have been received or accounted for by R8(b).

**R10 (shared landing slots).** Landing slots are assigned in one fixed order, by player index, and each observer includes its own prospective landing: when predicting player $X$'s landing, the predicted landings of players with a larger index are ignored.

### 4.3 Formal description of protocol $\mathcal{A}_5$

The protocol is $\mathcal{A}_5 = \langle I, S, R \rangle$. In the configuration of $A_j$ we write myind, dir, Used and, for each $K \neq j$, $\mathrm{pos}_K$, $\mathrm{dir}_K$, $\mathrm{last}_K$, $\mathrm{pre}_K$ (the largest $\ell$ such that all of $K$'s messages $1, \ldots, \ell$ are received or accounted for), $\mathrm{fwd}_K$ and $\mathrm{hold}_K$ (a map from pads to message numbers). Let $\mathrm{buf}_K = \mathrm{fwd}_K \cup \mathrm{dom}(\mathrm{hold}_K)$, $\mathrm{Occ} = \mathrm{Used} \cup \{\mathrm{myind}\} \cup \bigcup_K (\{\mathrm{pos}_K\} \cup \mathrm{buf}_K)$, and let $\mathrm{nodes}(n)$ be the BSP split points in breadth-first order. The helpers and the procedures $I, S, R$ are given in Fig. 2.

### 4.4 Security

The proof for $\mathcal{A}_4$ rests on the PU-buffer property (Lemma 3). With four players the pads $K$ uses after a jump depend on the occupancy $K$ sees, which an observer can only predict from its own view, so we isolate the property that is needed.

**Definition 5 (Prediction agreement).** A run of $\mathcal{A}_5$ satisfies *prediction agreement* if, for all players $A_j \neq A_K$: (i) whenever $K$'s $s$-th message is undelivered to $A_j$ at a $\mathsf{Send}$ of $A_j$ and $s > \mathrm{last}_j(K)$, the pad used by $K$'s $s$-th $\mathsf{Send}$ is the $(s - \mathrm{last}_j(K))$-th pad of $\mathrm{fwd}_j(K)$; and (ii) whenever R8(b) marks pads for skipped messages of $K$, the marked pads include those that $K$ used.

While all players walk, prediction agreement holds automatically, since a walk step of $K$ is determined by $\mathrm{pos}_j(K)$ exactly as in $\mathcal{A}_4$; condition (ii) holds in the third case of R8(b) whenever $K$ jumped at most once among the skipped sends.

**Lemma 5.** *Let $\mathcal{P} \in \mathcal{U}^{4,n}_{\mathcal{A}_5,d}$ have a run satisfying prediction agreement. When a* $\mathsf{Send}$ *of $A_j$ starts, every index used by some $A_K$, $K \neq j$, and still undelivered to $A_j$ lies in $\mathit{Used}_j \cup \mathit{buf}_j(K)$.*

*Proof sketch.* Let $i$ be the index of $K$'s $s$-th message and $\ell = \mathrm{last}_j(K)$; $s \neq \ell$ since message $\ell$ was received. If $s < \ell$, the first message of $K$ with a number above $s$ that reached $A_j$ skipped $s$, so by (ii) R8(b) put $i$ into $\mathrm{Used}_j$. If $s > \ell$, messages $\ell + 1, \ldots, s$ of $K$ were all sent and none delivered, so $s - \ell \leq d$ by definition of $\mathcal{U}^{4,n}_{\mathcal{A}_5,d}$, and by (i) $i$ is the $(s - \ell)$-th pad of $\mathrm{fwd}_j(K) \subseteq \mathrm{buf}_j(K)$. □

```
Helper Jump(dir, W) /* R2, R3 */:
for each M in nodes(n), M + 1 ≤ n:
    if M, M + 1 ∈ W or slot=(dir= +1 ? M + 1 : M)∈ W: continue
    front, back = free runs beyond the node in directions dir, −dir
      (back side: pads reserved only for direction −dir are free)
    if front > 2d and back > 2d: return slot
return ⊥
```

```
Helper Predict(X) /* R4–R6, R10 */:
p = position of X; path = (); repeat d times:
    W = Used ∪{myind} ∪ {pos_K, buf_K : K ≠ X}∪ path
    if p+dir_X ∈ [n] \ W: p = p+dir_X
    else: p = Jump(dir_X, W minus landing claims of players Y > X,
        plus A_j's own landing claim if j < X); if p =⊥: break
    append p to path
return path
```

```
Procedure I (input id):
A_1: myind= 1, dir= +1; A_2: myind= ⌊n/2⌋, dir= −1; A_3: myind=
⌊n/2⌋ + 1, dir= +1; A_4: myind= n, dir= −1
Used = ∅; for K ≠ id: pos_K, dir_K = K's start, last_K = pre_K = 0,
hold_K = ∅, fwd_K = Predict(K)
```

```
Procedure S (input Π):
if myind+dir ∈ [n]\Occ: myind = myind+dir /* walk, R1 */
else: t = Jump(dir, Occ); if t =⊥: return ⊥, halt; myind = t /* R2
*/
Used = Used ∪{myind}; c = c + 1; return: myind, c
```

```
Procedure R (input Π and ⟨K, i, c⟩):
Used = Used ∪{i}
if c > last_K: /* R8(a): older messages stop here */
    if c > last_K + 1: mark the skipped pads as in R8(b)
    last_K = c; pos_K = i
update pre_K; drop from hold_K pads held for numbers ≤ pre_K /* R9 */
for X = A_1, ..., A_4: /* R7, R9, R10 */
  if X = A_j: own landing claim = landing part of Predict(A_j)
  else: fwd_X = Predict(X); add its pads to hold_X
```

**Fig. 2.** Formal description of protocol $\mathcal{A}_5$ (player $A_j$).

**Theorem 5.** *Protocol $\mathcal{A}_5$ is secure on the ACPs in $\mathcal{U}^{4,n}_{\mathcal{A}_5,d}$ whose runs satisfy prediction agreement.*

*Proof sketch.* Take two successful Send events. If both are from $A_j$, the second avoids $\mathrm{Used}_j$, which contains the first. Otherwise let them be from $A_K$ and then $A_j$. If $K$'s message was delivered to $A_j$, its index is in $\mathrm{Used}_j$; if not, it is in $\mathrm{Used}_j \cup \mathrm{buf}_j(K)$ by Lemma 5. Procedure $S$ never returns an index in Occ (R1), so the indices differ. □

Rules R3, R6, R9 and R10 serve to make prediction agreement hold. It is not implied by membership in $\mathcal{U}^{4,n}_{\mathcal{A}_5,d}$: in rare edge cases two players with different views predict different landings for a jump. In such a case the protocol halts, and the remaining pads count as wasted in all the efficiency figures below.

### 4.5 Efficiency

We follow the accounting used for the earlier protocols: we bound the room a landing needs, the free space a halting player can leave behind, and the number of separate free regions. Call a *free interval* a maximal run of pads that are unused and are not a starting pad or a player's current position.

**Lemma 6 (Room for a landing).** *If node $\langle M, M+1\rangle$ is valid for a player, the $2d+1$ pads on each side of the node are free in its view. Hence a free interval of width $w$ that contains both pads of a valid node satisfies $w > 4d+3$.*

*Proof.* Let the interval be bounded by occupied pads $L < M$ and $R > M+1$, so $w = R - L - 1$. More than $2d$ free pads on the left means $M - L - 1 > 2d$, i.e. $M > L + 2d + 1$; on the right, $R - (M+1) - 1 > 2d$, i.e. $M < R - 2d - 2$. An integer $M$ exists only if $(R-2d-2)-(L+2d+1) > 1$, that is $R-L-1 > 4d+3$. □

Compared with the condition $w > 2d+3$ of the paired protocol, a single lander needs twice the cushion: up to $d$ unheard pads of a player approaching the node, and up to $d$ of its own pads that the others have not heard of.

**Lemma 7 (Halting leaves only short intervals).** *Let $d \le \lfloor n/4 \rfloor - 1$. When a player halts, every maximal run of pads that are free in its view (not in Occ) has fewer than $16d+19$ pads.*

*Proof sketch.* The depth-$k$ intervals of the BSP tree, separated by single split points of lower depth, partition $[n]$, and an induction on $k$ shows that their lengths differ by at most one; a child of an interval of length $x$ has length $\lfloor (x-1)/2 \rfloor$ or $\lceil (x-1)/2 \rceil$. Let $k$ be the largest depth whose shortest interval has length at least $4d+4$. Then the shortest depth-$(k+1)$ interval has length at most $4d+3$, so every depth-$k$ interval has length at most $8d+9$, and a free run of at least $2(8d+9)+1 = 16d+19$ pads contains a whole depth-$k$ interval $[l, r]$. Its split point gives a node with $\lfloor (r-l)/2 \rfloor \ge 2d+1$ free pads on the left and $\lceil (r-l)/2 \rceil - 1 \ge 2d+1$ on the right, which is valid, so the player would not halt. □

**Lemma 8 (Free intervals and jumps).** *After $J$ jumps there are at most $J+2$ free intervals.*

*Proof.* Initially the free intervals are the two arena interiors. A walk step uses a pad at the end of an interval, which shrinks or removes it but never splits it. A landing uses one pad inside an interval and splits it into at most two, so each jump adds at most one interval. □

**Theorem 6.** *Consider a run of $\mathcal{A}_5$ in which every player halts, with $J$ jumps in total, and let $\rho$ be the number of unused pads that the last player to halt counts as occupied at that moment (reserved or held for others, or marked by R8(b)). At most $4 + \rho + (J + \rho + 2)(16d + 18)$ pads are wasted, so*

$$\alpha \;\ge\; 1 - \frac{4 + \rho + (J + \rho + 2)(16d + 18)}{n}.$$

*Proof.* No pad is used after the last player halts. The unused pads are the four starting pads and the pads of the free intervals at that moment. A free interval containing $r$ of the $\rho$ pads splits, in the halting player's view, into at most $r+1$ runs, each of at most $16d+18$ pads by Lemma 7, so it has at most $r+(r+1)(16d+18)$ pads. Summing over the at most $J+2$ intervals of Lemma 8 gives the bound. □

Theorem 6 gives $\alpha \geq 1 - O(d\,(J+\rho)/n)$. The constant is worst case per interval and the bound is loose in practice, but it shows where the waste comes from: $O(d)$ pads per jump and per pad of reservation slack. In 2,894 instrumented runs ($n \leq 2048$, $d \leq 16$), $J$ grew roughly like $\log_2(n/(4d+4))$: for $d=2$ its mean rose from 1.0 at $n=64$ to 6.5 at $n=2048$ (maximum 19 over all runs); $\rho$ stayed $O(d)$, about $1.3d$ on average; and the waste never exceeded 7% of the bound of Theorem 6. If $J = O(\log(n/d))$ and $\rho = O(d)$, the theorem gives $\alpha \geq 1 - O(d(\log(n/d)+d)/n)$, which tends to 1 for fixed $d$; for the paired protocol, the corresponding bound on $J$ follows from a halving argument, while for individual jumps we leave it open.

### 4.6 Experimental evaluation

We implemented $\mathcal{A}_5$ together with an event-level simulator of the model and an interactive visualizer (Fig. 1). The simulator enforces the undelivery bound of $\mathcal{U}^{4,n}_{\mathcal{A}_5,d}$ exactly: before every Send, at most $d$ messages addressed to the sender are undelivered.

**Setup.** The delivery adversary is one of nine strategies: synchronous (*instant*); maximal delay in order (*worst*); random early delivery with probability 0.5 or 0.1 (*random*, *adversarial*); releasing the newest, or a random, pending message when a queue is full (*lifo*, *randcap*); random order (*shuffle*); and delaying all messages from, or to, one player (*target-sender*, *target-recv*). The schedule is one of twelve sender orders, from uniformly random and round robin to one player or one arena sending most of the time (*skew*, *arena-skew*), one or two silent players, bursts, and orders that make players meet as often as possible (*greedy-gap*). We ran every combination for $n=256$, $d=2$ with 100 seeds, and for pad sizes from 16 to 2048 and buffer bounds from 1 up to $\lfloor n/4 \rfloor - 1$ with fewer seeds: 35,964 runs in total. Every run counts: a run that halts in one of the rare edge cases of Section 4.4 counts its remaining pads as wasted.

**Pad size and buffer bound.** Figure 3(a) and Table 1 show that for fixed $d$ the waste is a small number of pads that grows slowly with $n$, so the efficiency ratio approaches 1: for $d=2$ the mean is 0.963 at $n=256$ (about 9 pads wasted) and 0.994 at $n=2048$ (about 13), and under synchronous delivery even the worst run at $n=2048$ uses 0.992 of the pad. For small $n$ the four starting pads and the clearance around landings are a large fraction of the pad. Figure 3(b) shows the cost of a larger buffer bound: each extra undelivered message enlarges the reserved buffers and the clearance a landing needs (Lemma 6). As $d$ approaches $n/4$ no landing is possible and each player is confined to its quarter of the pad, the efficiency of the trivial protocol $\mathcal{A}_1$.

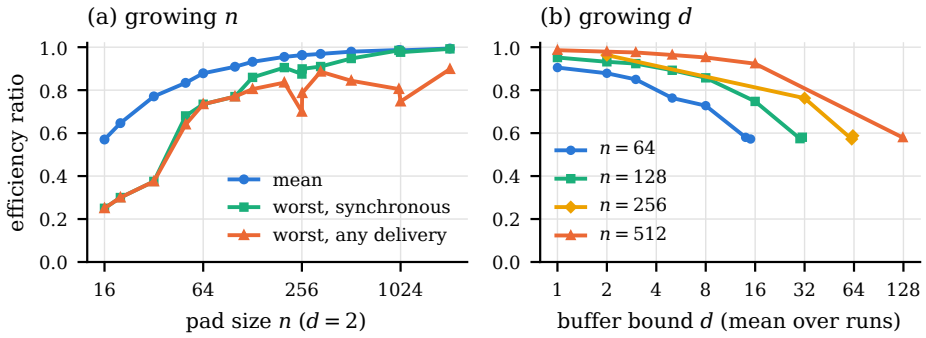



**Fig. 3.** Efficiency ratio (a) as a function of $n$ for $d = 2$, and (b) as a function of $d$ for four pad sizes, over all delivery adversaries and sender orders.

**Table 1.** Efficiency ratio by pad size $n$ and buffer bound $d$, over all delivery adversaries and sender orders: the mean, the worst run under synchronous delivery, and the worst run overall.

| $n$ | $d$ | runs | mean | worst (sync.) | worst (any) |
|---|---|---|---|---|---|
| 64 | 2 | 324 | 0.879 | 0.734 | 0.734 |
| 128 | 2 | 324 | 0.932 | 0.859 | 0.805 |
| 256 | 2 | 10,800 | 0.963 | 0.875 | 0.699 |
| 512 | 2 | 324 | 0.979 | 0.947 | 0.844 |
| 1024 | 2 | 90 | 0.985 | 0.977 | 0.747 |
| 2048 | 2 | 90 | 0.994 | 0.992 | 0.898 |
| 128 | 8 | 324 | 0.857 | 0.750 | 0.625 |
| 512 | 8 | 324 | 0.952 | 0.852 | 0.744 |
| 2048 | 8 | 90 | 0.984 | 0.969 | 0.889 |
| 512 | 16 | 324 | 0.923 | 0.682 | 0.682 |
| 256 | 32 | 360 | 0.763 | 0.734 | 0.734 |
| 256 | 63 | 360 | 0.588 | 0.492 | 0.492 |

**Delivery and sender order.** Figure 4 breaks the $n = 256$, $d = 2$ runs down. The mean efficiency hardly depends on the delivery adversary (between 0.960 and 0.967), because the reserved buffers cost the same pads whether or not the adversary uses the slack. The sender order matters more: when one player or arena dominates, or players are silent, the active players must jump into small regions, while flooding orders let players walk long distances. Silent players do not waste their quarter of the pad, since the others jump into it.

## 5 New OTPAC Protocols for $m = 2^l$ Parties

Intro paragraph (what is presented in this section)
Main theorem for this section
Remarks to the theorem
Informal description of the main result

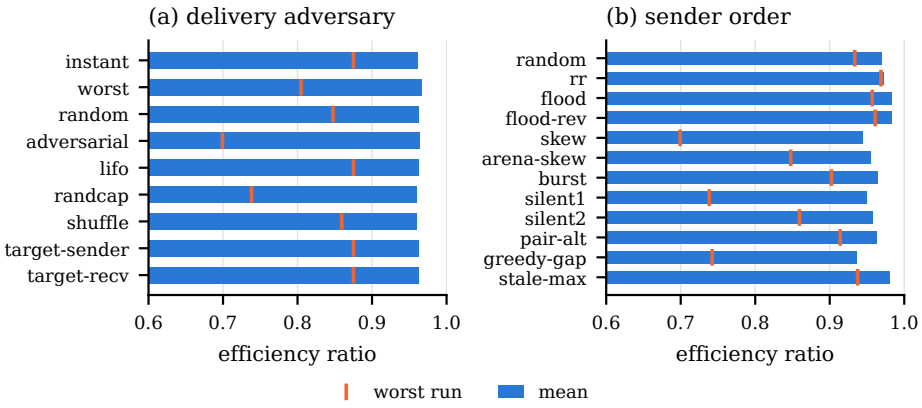



**Fig. 4.** Efficiency ratio at $n = 256$, $d = 2$ by (a) delivery adversary and (b) sender order (1,200 and 900 runs per bar).

Formal description of the main result
Proof of the two protocol properties: security and efficiency ratio

## References


1. R. Anderson and H. Woll, *Algorithms for the Certified Write-All Problem*, in SIAM Journal on Computing, vol. 139(1), 1997, p. 1–16.
2. James Aspnes, *Randomized protocols for asynchronous consensus*, Invited survey paper for Distributed Computing, PODC 20th anniversary issue. 2002.
3. J. Buss, P. Kanellakis, P. Ragde, and A. Shvartsman, *Parallel Algorithms for Process Failures and Delays*, in Journal of Algorithms, vol. 20(196), 1997, p. 45–86.
4. Ran Canetti, *Studies in Secure Multiparty Computation and Applications*, Ph.D. Thesis, Weizmann Institute of Science, Israel, 1995.
5. B. Chlebus, R. De Prisco, and A. Shvartsman, *Performing Tasks on Synchronous Restartable Message-Passing Processors*, in Distributed Computing, vol. 14, 2001, p. 49–64.
6. R. De Prisco, A. Mayer and M. Yung, *Time-Optimal Message-Efficient Work-Performance in the Presence of Faults*, in Proc. of PODC 94.
7. D. Z. Du and F. K. Hwang, *Combinatorial Group Testing and its Applications,* World Scientific, Singapore, 1993.
8. C. Dwork, J. Halpern and O. Waarts, *Performing Work Efficiently in the Presence of Faults*, in SIAM Journal on Computing, vol. 27, 1998, p. 1457–1491.
9. P. Erdos, P. Frankl and Z. Furedi, *Families of finite sets in which no set is covered by the union of r others,* in Israeli Journal of Mathematics, 51: 79–89, 1985.
10. Michael J. Fischer, Nancy A. Lynch and Mike Paterson, *Impossibility of Distributed Consensus with One Faulty Process*, JACM 32(2): 374-382 (1985)
11. Oded Goldreich, *Foundations of Cryptography (Fragments of a Book)*, 1995. Electronic publication: http://www.eccc.uni-trier.de/eccc/info/ECCC-Books/eccc-books.html (Electronic Colloquium on Computational Complexity).

12. J. Groote, W. Hesselink, S. Mauw, and R. Vermeulen, *An Algorithm for the Asynchronous Write-All Problem based on Process Collision*, in Distributed Computing, vol. 14(2), 2001, p. 75–81.
13. P. Kanellakis and A. Shvartsman, *Efficient Parallel Algorithms Can Be Made Robust*, in Distributed Computing, vol. 5(4), 1992, p. 201–217.
14. D. Kowalski and A. Shvartsman, *Performing Work with Asynchronous Processors: Message-Delay-Sensitive Bounds*, in Proc. of PODC 2003, p. 265–274
15. Leslie Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System* CACM 21(7): 558-565 (1978)
16. Michael Luby, *Pseudorandomness and Cryptographic Applications*, Princeton University Press, 1996.
17. C. E. Shannon, *A Mathematical Theory of Communication*, The Bell System Technical Journal 27, 1948, 379-423, 623-656.
18. J. Sussman and K. Marzullo, *The Bancomat Problem: An Example of Resource Allocation in a Partitionable Asynchronous System*, Proc. of PODC 98.
19. G. Vernam, *Secret Signaling Systems*, US Patent, 1919.